

LABSHEET 6

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Load dataset
df = pd.read_csv("House price prdict.csv")

# Display first 5 rows
print(df.head())
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	\
0	13300000	7420	4	2	3	yes	no	no	
1	12250000	8960	4	4	4	yes	no	no	
2	12250000	9960	3	2	2	yes	no	yes	
3	12215000	7500	4	2	2	yes	no	yes	
4	11410000	7420	4	1	2	yes	yes	yes	

	hotwaterheating	airconditioning	parking	prefarea	furnishingstatus
0	no	yes	2	yes	furnished
1	no	yes	3	no	furnished
2	no	no	2	yes	semi-furnished
3	no	yes	3	yes	furnished
4	no	yes	2	no	furnished

```
print("Shape of dataset:", df.shape)
```

```
print("\nColumn names:")
print(df.columns)
```

```
print("\nData types:")
print(df.dtypes)
```

```
print("\nMissing values:")
print(df.isnull().sum())
```

```
Shape of dataset: (545, 13)
```

```
Column names:
Index(['price', 'area', 'bedrooms', 'bathrooms', 'stories', 'mainroad',
       'guestroom', 'basement', 'hotwaterheating', 'airconditioning',
       'parking', 'prefarea', 'furnishingstatus'],
      dtype='object')
```

```
Data types:
price          int64
area           int64
bedrooms       int64
bathrooms      int64
stories        int64
mainroad       object
guestroom      object
basement       object
hotwaterheating object
airconditioning object
parking        int64
prefarea       object
furnishingstatus object
dtype: object
```

```
Missing values:
price          0
area           0
bedrooms       0
bathrooms      0
stories        0
mainroad       0
guestroom      0
basement       0
hotwaterheating 0
airconditioning 0
parking        0
```

```
prefarea      0
furnishingstatus  0
dtype: int64
```

Data Preprocessing

```
# Convert categorical columns into numerical values
df_encoded = pd.get_dummies(df, drop_first=True)
```

```
print("Encoded dataset:")
print(df_encoded.head())
```

```
print("\nNew shape:", df_encoded.shape)
```

Encoded dataset:

	price	area	bedrooms	bathrooms	stories	parking	mainroad_yes \
0	13300000	7420	4	2	3	2	True
1	12250000	8960	4	4	4	3	True
2	12250000	9960	3	2	2	2	True
3	12215000	7500	4	2	2	3	True
4	11410000	7420	4	1	2	2	True

	guestroom_yes	basement_yes	hotwaterheating_yes	airconditioning_yes \
0	False	False	False	True
1	False	False	False	True
2	False	True	False	False
3	False	True	False	True
4	True	True	False	True

	prefarea_yes	furnishingstatus_semi-furnished	furnishingstatus_unfurnished
0	True	False	False
1	False	False	False
2	True	True	False
3	True	False	False
4	False	False	False

New shape: (545, 14)

Select Features and Target

```
# Target variable
y = df_encoded["price"]

# Independent variables
X = df_encoded.drop("price", axis=1)
```

```
print("Features:")
print(X.columns)
```

```
print("\nTarget:")
print(y.name)
```

Features:

```
Index(['area', 'bedrooms', 'bathrooms', 'stories', 'parking', 'mainroad_yes',
       'guestroom_yes', 'basement_yes', 'hotwaterheating_yes',
       'airconditioning_yes', 'prefarea_yes',
       'furnishingstatus_semi-furnished', 'furnishingstatus_unfurnished'],
      dtype='object')
```

Target:
price

Split Dataset — 80:20

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)
```

```
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

Training data: (436, 13)
Testing data: (109, 13)

Train Linear Regression Model

```

model = LinearRegression()

model.fit(X_train, y_train)

print("Linear Regression model trained successfully!")

Linear Regression model trained successfully!

```

Predict House Prices

```

y_pred = model.predict(X_test)

print("Actual Prices:")
print(y_test.values[:10])

print("\nPredicted Prices:")
print(y_pred[:10])

Actual Prices:
[4060000 6650000 3710000 6440000 2800000 4900000 5250000 4543000 2450000
 3353000]

Predicted Prices:
[5164653.90033967 7224722.29802166 3109863.24240338 4612075.32722559
 3294646.25725955 3532275.09556558 5611774.56836476 6368145.98732718
 2722856.95689985 2629405.61585782]

```

Actual vs Predicted Visualization

```

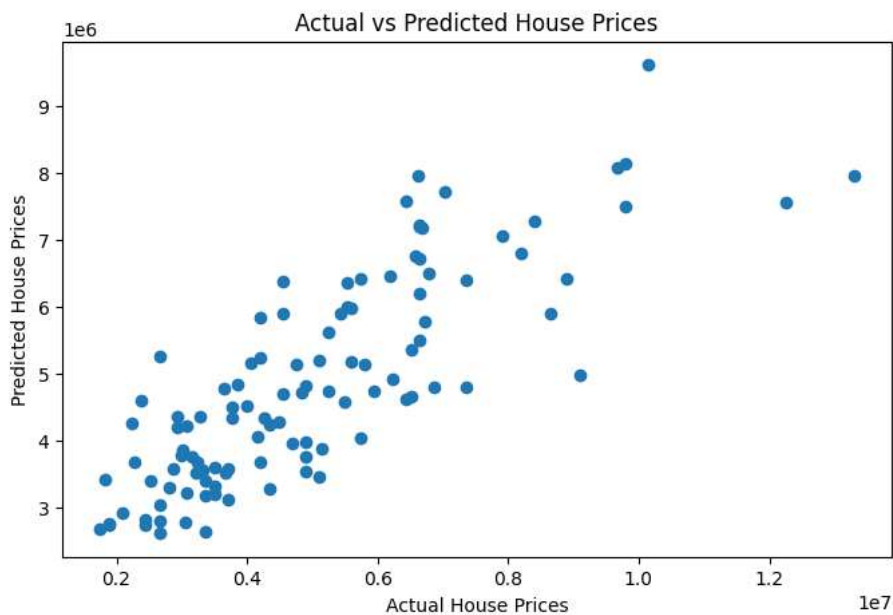
plt.figure(figsize=(8,5))

plt.scatter(y_test, y_pred)

plt.xlabel("Actual House Prices")
plt.ylabel("Predicted House Prices")
plt.title("Actual vs Predicted House Prices")

plt.show()

```



Evaluate Model

```

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("Model Performance")

```

```
print("-----")
print("Mean Absolute Error (MAE):", mae)
print("Mean Squared Error (MSE):", mse)
print("Root Mean Squared Error (RMSE):", rmse)
print("R2 Score:", r2)
```

Model Performance

Mean Absolute Error (MAE): 970043.4039201636
Mean Squared Error (MSE): 1754318687330.6638
Root Mean Squared Error (RMSE): 1324506.9600914386
R² Score: 0.6529242642153184

Regression Coefficients

```
coefficients = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_
})

print(coefficients)
```

	Feature	Coefficient
0	area	2.359688e+02
1	bedrooms	7.677870e+04
2	bathrooms	1.094445e+06
3	stories	4.074766e+05
4	parking	2.248419e+05
5	mainroad_yes	3.679199e+05
6	guestroom_yes	2.316100e+05
7	basement_yes	3.902512e+05
8	hotwaterheating_yes	6.846499e+05
9	airconditioning_yes	7.914267e+05
10	prefarea_yes	6.298906e+05
11	furnishingstatus_semi-furnished	-1.268818e+05
12	furnishingstatus_unfurnished	-4.136451e+05

T **B** *I* $\langle \rangle$ “ ” — Ψ 😊 [Close](#)

Regression Line

Regression Line

```
# Using area as the main feature for simple regression visualization

X_area = df[["area"]]
y_price = df["price"]

X_train_area, X_test_area, y_train_area, y_test_area = train_test_split(
    X_area, y_price, test_size=0.20, random_state=42
)

area_model = LinearRegression()
area_model.fit(X_train_area, y_train_area)

y_area_pred = area_model.predict(X_test_area)

plt.figure(figsize=(8,5))

plt.scatter(X_test_area, y_test_area, label="Actual")
plt.plot(X_test_area, y_area_pred, label="Regression Line")

plt.xlabel("Area")
plt.ylabel("House Price")
plt.title("House Price vs Area - Linear Regression")

plt.legend()
plt.show()
```

